

Advanced Natural Language Processing

Flash Attention? Ring Attention?

What design choices in the model? What tradeoffs

Phrase based Statistical Machine Translation $\leftarrow$ maximise IP of phrases of L_1 translated into L_2 while also ensuring grammar of L_2 is preserved.

Language is not a time series, hence Parallel Neural Networks like Transformers are better at Knowledge Representation.

Mauritius Creole - French & Awedhi

Language is a dialect with an army & a navy

Aparajitha's Class

Intro to Transformers.

Recap sess - One hot, Tf-idf, word2vec/GLoVe, BERT, LLM Embeddings

Over time, context $\uparrow$, ~~and~~ ambiguity $\downarrow$, density $\uparrow$ and complexity $\uparrow$

Gradients & Backprop - just refer iNLP notes.

Gradient Descent = $\theta - \eta \nabla_{\theta} J(\theta; x, y)$

$\hookrightarrow$ learning rate = hyperparam

$\hookrightarrow$ Gradient Clipping

$\hookrightarrow$ Regularisation

$\hookrightarrow$ Normalisation

$\hookrightarrow$ Temperature.

Back prop $\rightarrow$ Forward & Backward pass

$\hookrightarrow$ Error calculation & gradient propagation

Problems with RNN

- $\hookrightarrow$ Long distance relationship failure $\therefore$ Anaphora, Coreference, Causal Temporal Discourse structures.
- $\hookrightarrow$ Finite Context
- $\hookrightarrow$ Computationally hard
- $\hookrightarrow$ Vanishing Gradients
- $\hookrightarrow$ Exploding Gradients

Bahdanau - Attention - Simple attention $\rightarrow$ Soft attention in INLP

Attention aids interpretability

Self Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Query $\sim$ Question asked (about relatedness)

Key $\sim$ lookup Query to the lookup table (Query here $\neq Q$)

Value $\sim$ measure of wt allocated to the word.

MHA $\leftarrow$ multi-head

Positional Encoding

Masking for decoder, set to $-\infty$ before softmax

L_o Layer Norm

Why this specific fn for attention

Let q_i be the token asking a question (what is the most relevant token)

$\sim$ relevance scorer.

Let us begin with $\max(f(q_i; k_j))$

No derivation, but design choices exist

Each token has an embedding PE_i associated with

$$PE(i) \rightarrow q_i \quad PE(j) \rightarrow k_j$$

We write a fn approximator for $f()$

multilayer perceptron $\rightarrow$ Neural Network

$$MLP(q_i, k_j)$$

$\Downarrow$

$MLP_2(q_i, k_j, W_1, W_2) \rightarrow$ returns a single scalar val.

$\downarrow$

but one of these is not sufficient, since there are multiple relationships possible b/w words, so we need multiple of these (heads)

But this is computationally bad.

So replace it with a QK^T ($q_i \cdot k_j$ rather)

But currently q_i & k_j are just embeddings, hence the dot product is fast but not complete.

We could do something like $q_i \cdot W \cdot k_j$ where W is learnable. (Why does it look so much like a decomposition?)

Say $PE(i)$ are d_i -dimensioned. We project this down to d_k dimensions where $d \sim 8d_k$ (std. since q & heads) by multiplying a matrix W $d_i \times d_k$.

But we need different W_q & W_k for every word. These encode the relevance of different terms.

Now the dot product encodes data, hence QK^T is now viable.

$$QK^T = \forall i, j, q_i \cdot k_j$$

Kernel fns - maps data such that a single classmate
line or plane can separate the data

Date _____
Page _____

Pass this through a softmax fn

$$\text{Softmax}(\vec{v}) = \left[\frac{e^{v_i}}{\sum_j e^{v_j}} \right] + i$$

$\text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right)$ finishes the calculation of attention
↳ empirically set

But this just gives a score vector where the most imp position has the highest value.

We multiply $\vec{A}$ with V (which is a third projection of PE with W_V)

Context representation of word: $c_i = \sum_j \text{attention}(i, j) V_j$

V is supposed to carry the meanings of the words

Residuals happen b/w every sublayer through add & norm

NN projects ~~sub~~ embeddings into a higher or lower dimension space that is completely different subspace than the original embedding space.

Say you have $x_1, \dots, x_n$

Let $\exists$ a neuron that takes these inputs & performs

$$\sigma(A \cdot \vec{x} + B)$$

All of these ^{neurons} generate hyperplanes

Say the

कक्षा में काम को करने से सम्प्रवर्ण की के लिए
आपादन से संबंध का के की अधिकरण में पर ऊपर संबंधन से जो

classmate
Date _____
Page _____

We use a σ since we don't care about the magnitude of ~~bits~~ in the accuracy if it is too high or too low. We only care what the ~~smaller~~ smaller accuracies are.

We use multiple layers since one layer is not enough to learn all the information from an input.

Scaled Dot Product Attention

$$\text{softmax} \left(\frac{(XW_i^Q)(XW_i^K)^T}{\sqrt{d_k}} \right) (XW_i^V)$$

MHA

Multi-Head $(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$

head_i = Attention (XW_i^Q, XW_i^K, XW_i^V)

Transformers are traditionally post norm (after FFN) but there are post-pre norm machines as well.

Do transformers solve long contexts?

Traditionally, yes.

↳ But new ones like code understanding, long/multi-document QA- Performance degrades in the middle of the context. Highest ~~begin~~ Highest at beginning & end. Likely due to presence of summaries & introduction. Most likely cause humans themselves attend in such a fashion.

Tokenisation (Again ?!)

Space separation? Bad bad

Separate case markers? - Stemming / Lemmatization

Tokens $\rightarrow$ atomic units of text

Ref: NLP

Token $\rightarrow$ id for a piece of input

Word Piece:-

Initialisation works the same way as BPE.
But during tokenisation, it uses a greedy longest match strategy instead of a merges dict order.

Embedding size in modern models $\sim 512-1024$, not 100-200!

$\rightarrow$ $\langle \text{UNK} \rangle$ for unknown characters

Unigram Language Modelling $\rightarrow$ Check Markovian bit-minimising loss.

Expectation Maximisation:- Assume a hypothesis, observe a better fit, switch to that & repeat to maximise likelihood of data. For example:- k-means.

ByteBPE my govt.

Parity Aware BPE:-

No use? At least practically

No Tokenisation

Great for generation

$\hookrightarrow$ Byte or Character level models

$\hookrightarrow$ Continuous Input Encoders

$\hookrightarrow$ Hybrid Models

RLT:-

Patches of n bytes, create a latent space to create a specific representation for the patches.

Check slides:-

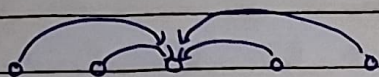
Passivisation & Topicalisation causes
object fronting in English.

classmate

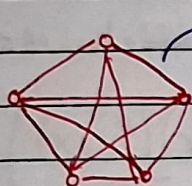
Date

Page

Word Position



Can be visualised graphically where every word is connected to every other word (bidirectionally)



→ A fully connected component.

A collection of these fully connected components can encapsulate various sparser graphs (syntactic, semantic, etc.). Attention scores scale the wt of the edges (set some $\rightarrow 0$ etc)

But graphs lose information about word ordering & disallowed bindings.

↳ fast green car vs green fast car

LWG (Local Word Groups) mitigate these issues

↳ rigid; but outside this languages become more free.

राम घर गया vs राम गया घर vs गया राम घर
vs घर गया राम

∃ a canonical (preferred) form $\rightarrow$ verb final.

Word order also marks causality

PE hacks the positional agnostic models with a sense of position

Absolute PE:-

We just need to add the information of the position to the word. We need to encode only the seq-len worth of positions (distinct vectors for all positions)

Create an ^{pos} embedding matrix of the embedding matrix.

not really, it does learn relative positions
but is big data dependent.

But this lacks relative positions. It just remembers the position vectors. Also fails to predict unseen positions ($> \text{seq len}$)

Sinusoidal PE :-

The embedding fn handles "relative positions".

Any offset PE_{pos+k} is a linear fn of PE_{pos} .

This is also added to the word embedding

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Output of sin & cos is bounded $[-1, 1]$

Also differentiable:

The complex argument is so that the same phase PE doesn't appear for a long time.

This can extrapolate to longer sequences.

The whole model is position independent except PE itself

No learnable parameters :(

PEs can only be extrapolated to some arbitrary distance, not ∞

Relative PE :-

Pairwise embeddings instead of individual embeddings.

Part of attention since it is sentence dependent.

Calculated b/w two vectors.

TS :- Δpos Single scalar is added to the corresponding logit used for computing attention weights
Shared Δpos across all layers.
Each head uses a different learned parameters $\Delta pos (ish)$

Scales logarithmically [32 → 128 etc]

Cumulative effect possible.

Complex implementation; can't cache ^{changes for every token} hence slow

Rotary PE:-

Relative pos is encoded in the angular difference b/w the two words' embeddings.

Rotates the (affine-transformed) word embedding by the Δpos + some angular step.

Consider every vector as a collection of 2-d vectors

Rotation Matrix

$$R_{\theta, m}^d = \begin{bmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \dots & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \dots & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \dots & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \cos m\theta_{d/2} \end{bmatrix} \quad \theta_{d/2}$$

$$f_q(x_n, m) = (W_q x_n) e^{im\theta}$$

$$f_k(x_n, n) = (W_k x_n) e^{in\theta}$$

$$g(x_m, x_n, m-n) = \text{Re} \left[\overbrace{(W_q x_m)}^q \overbrace{(W_k x_n)}^k e^{i(m-n)\theta} \right] \quad \text{relative pos.}$$

This is infinitely extensible.

Inter token dependency $\propto \frac{1}{\Delta pos}$

Interpretable.

Try implementing RoPE

classmate

Date

Page

but is complex implementation-wise (nameable)

No PE:-

Doesn't say it all

∃ a causal mask which gives autoregressive model an implicit sense of order.

Works very well for GPT like model (nothing else!)

Predecessor RoPE

Causal Mask $\leftarrow$ token at t can only attend to tokens at $s \leq t$

No additional computations!

Better at generalising long distances

Theoretically learns both abs & rel pos emb.

Pencil time T_{WT}

Attention:-

Scaled Dot Product Attention

quadratic attention

Multi Headed Attention $\rightarrow$ multiple ways of looking at content

$\hookrightarrow$ Parallelisable but $O(n^2)$, redundant heads, slow inference

& very large KV as cache.

During inference, generating every token results in computing QK^T matrices, but every new one has the previous ones in it.

Hence we cache KV. Good Speedup, but complex to code & has increased memory requirements

Have a single K & V for all Q s.

Longformer Attention (2020)

Localised Attention + Globalised Attention.

Sliding Window Attention:

Have a fixed window w & each token attends only to $w/2$ tokens on each side.

Dilated Sliding Window Attention

Pick random other tokens to attend to as well.

Global Attention

Some tokens attend to all tokens

↳ $\langle \text{CLS} \rangle$

Amortised $O(n)$ time.

They tried to increase w with layer height.

Multi Query Attention (2019)

Have ~~only~~ one K & one V for all Q s

Loses information - performance degradation

Reduces quality

Lost information for inference speed

Also time spent loading $KV \uparrow$ in distributed computing.

Grouped Query Attention (2023)

Multiple heads are grouped and have the same KV matrices for each group.

How many groups? How do you assign them to heads?

Multi Headed Latent Attention (2024)

Compresses KV to a lower dimensional space

Doesn't lose ^{too much} information upon compression

$d_e \ll h \cdot d_h$ but d_e is also a "hyperplane".

Down project Q, K & V and train on the lower dimension space.

Requires pre-training from scratch, can't convert MHA to MLA.

Low rank compression of K & V . Q, K & V matrices, but you can pick & choose.

Why not have a small model? Over parameterisation is good! Lottery Ticket Hypothesis

Log-linear attention:-

Limit no. of things attended to to $\log N$.

Distance $\propto \frac{1}{\text{importance}}$ is the assumption.

no. of hidden states $\uparrow$ logarithmically
Not Measurable though.

VS

Sparse Attention:-

Attends to a window and to a stride.

Used in vision, fails at text.

Sparse Attention - Deepseek:-

Choose a few select tokens by Φ using a pre-attention head (sparse specialised, low dimension, one query) and picking top- k attention scores.

Occurs once at every layer.

Limitations:- Comp overhead, non-contiguous mem access, fixed stride = bad. (they don't exist :-))

Hybrid Attention - BigBird

Check paper.

Linear Attention:-

Quadratic cause multiplication with V in attention.

Can we replace this similarity fn?

Std softmax attention has

$$O_i = \frac{\sum_j \exp(q_i \cdot k_j) v_j}{\sum_j \exp(q_i \cdot k_j)}$$

This is n^2 in time.

We replace the $\exp(q \cdot k)$ with

$$\phi(q) \phi(k) ; \phi: d \rightarrow d'$$

ϕ is a kernel map-

$$O_i = \frac{\phi(q_i) \sum_j \phi(k_j) v_j}{\phi(q_i) \sum_j \phi(k_j)} \cdot \frac{\phi(q_i) s}{\phi(q_i) z}$$

$\hookrightarrow$ independent of q_i , precompute

Time is linear in N , memory is independent of N .

With a causal mask

$$s_t = s_{t-1} + \phi(k_t) v_t$$

$$z_t = z_{t-1} + \phi(k_t)$$

Hence the transformer is an RNN with a matrix-valued hidden state

This feature map ϕ needs to be +ve

But this loses representational power.

Normalisation:-

Why? - To ensure that our computation space is constrained to $[0, 1]$

let $y = ax_1 + bx_2$ where $\lambda = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i(a, b))^2$
(Mean Squared Error)

→ confocal ellipses

Based on the graph in the slides, one can say that the domain (x_1) $\gg$ domain (x_2)

We want this error landscape to be more spread out (to have concentric circles). We limit the domain of x & y .

This argument can be expanded to multiple dimensions.

- Increases the speed of convergence
- Reduces sensitivity to input features
- Reduces over-fitting → test data and train data need not have same domain for features
- Helps the model generalise better to features with smaller domains contribute to larger gradients.

If Area Under Receiver Operator Characteristics is too small → model has overfit

(i) Batch Norm:-

How do we choose what to compare features against
Normalise features across ~~between~~ samples. But we have many samples. So we chunk the samples into batches and normalise features across batches.

$$\mu_i = \frac{1}{B} \sum_{b=1}^B x_i^{(b)}$$

$$(\sigma_i)^2 = \frac{1}{B} \sum_{b=1}^B (x_i^{(b)} - \mu_i)^2$$

$$\hat{y}_i = \gamma_i \cdot \frac{x_i^{(b)} - \mu_i}{\sqrt{(\sigma_i)^2 + \epsilon}} + \beta_i$$

Problem:- Inconsistency in reliable sampling of population

(ii) Layer Norm:-

Normalise across features in a vector

$$\mu^{(b)} = \frac{1}{d} \sum_{i=1}^d x_i^{(b)}$$

$$\sigma_i^{(b)^2} = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$$

$$y_i = \gamma^{(b)} \frac{x_i^{(b)} - \mu}{\sqrt{\sigma_i^{(b)^2} + \epsilon}} + \beta^{(b)}$$

(iii) RMS Norm:-

Special case of Layer Norm where $\mu, \epsilon = 0$

$$y^{(b)} = \gamma^{(b)} \frac{x^{(b)}}{\sqrt{\frac{1}{d} \sum_{i=1}^d (x^{(b)})^2}} + \beta^{(b)}$$

β set to zero.

remove this for L2 Norm

(iv) Weight Norm:-

Separate wt vectors into magnitude & direction

Speeds up convergence

Applied per neuron. Helps keep weight space bounded.

$$y(x) = \frac{g}{\|v\|} v^T x + b$$

Residual Connections :-

~~$$x_{out} = w(x_{in} + output)$$~~

Helps propagate errors better

Supervised

vs

Unsupervised

$M = f(\text{Arch, Params}, \text{Data})$

$M = f(\text{Arch, Params}, \text{Data})$

but since the model is

an abstraction of the data

Model = a set of params & a fn that when given data, mimics the real world.

classmate

Date _____
Page _____

input class

$$M = f(\text{Arch}, \text{Params}) \leftarrow D(x, y)$$

$$\underset{P}{\operatorname{argmax}} P(y/x)$$

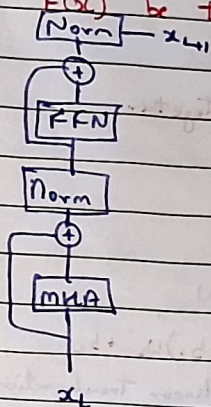
$$M = f(\text{Arch}, \text{Params}) \leftarrow D$$

$$\underset{P}{\operatorname{argmax}} P(D)$$

What is the parameter set that maximises expectation of the

Pre attention vs post attention norm are functionally equivalent except for skip connections.

Let $F(x)$ be the output of the attention block.



② Post Norm

$$\text{Norm}(x + F(x))$$

↓

Addition and Compression
Residual highway.

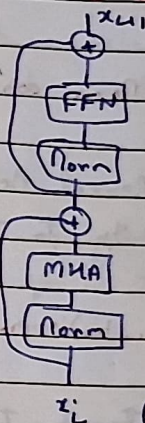
↳ Results in vanishing gradient.

↳ if done carefully → no loss of info.

Postnorm training is more unstable but prenorm leads to representation collapse. (each layer represents the same information)

Prenorm is good for interpretability though through cosine similarity.

Emergent features rise from prenorm.



① Pre Norm

$$x + F(\text{Norm}(x))$$

↓

Pure addition residual highway

Deep Norm:-

Scaled postnorm.

Adds a scaling term to the residual connections (can be learnt and can be set as a hyperparam)

$$x_{L+1} = \text{Norm}(\alpha x_L + F(x_L)).$$

↳ usually a linear function depth

Stabilises gradients across really deep network.

Hybrid Norm:-

Normalise @ K & V.

Post Norm & RMS Norm don't go well together.

Feed Forward Neural Networks

Adds non-linearity to the model.

↳ ReLU

$$\text{FFN}(x, w_1, w_2, b_1, b_2) = \max(0, xw_1 + b_1)w_2 + b_2$$

↳ linear transformations

Passes vector x through two learned matrices and then through a linear unit.

We want fns whose deriv derivatives are fns of the fn.
For ex- tanh, σ etc

A miss is as good as a mile

The error is not the dist of the point from the line but is rather the vertical distance from the line/hyperplane of the node.

The activation fn must compress the error range

Gated Linear Units

Gating:- x is projected by w through a σ producing a vector

of values $\in [0, 1]$. x is also projected by V which has an element wise product with $\sigma(xW_1)$ resulting in a filtered representation.

Gates are scalar multipliers to each cell in a vector.

Controls importance of each of the cells of the vector.

→ Gating Weight Matrix.

$$FFN_{GLU}(x, W_1, V, W_2) = (\sigma(xW_1) \otimes xV)W_2$$

We usually up-project using W_1 & V , but we could also down-project.

Gating allows creation of data-channels in large, over-parameterised models which results in noise reduction. ~~Relevance~~ Relevance / Saliency filter

Input dependent gating → richer representations through flexible, token / representation / region specific transformation.

Also helps in dynamic feature selection (in the up-projected hyperspace)

All GLU variants use three matrices, hence each matrix has $\frac{2^{rd}}{3}$ the normal dimensions

After $\sim 6.4B$ params, $\exists$ activation spikes in early & late layers due to GLU. Not an issue in fp32, but quantisation is

But FFNNs are static, dense and content unaware.

Every token goes through the full FFNN → inefficient and the FFNN must learn everything.

MoE:-

size of data

Scaling Laws:- No. of Params, $\propto$ quality, compute $\propto$ quality.

Instead of FFNNs, use multiple sub-models ("experts")

Partition input spec into diff regions at diff times, distribute compute, noisy interactions ↓

Routers learn to route data to specific top-k experts

Parallelisable, but unstable, complex & compute heavy.